

3D Gaussian Splatting 定式化

学生番号: 1030-38-7683 修士 1 回生 田中 知優

1 はじめに

複数視点から撮影された学習画像（真値画像）と、それぞれのカメラパラメータが与えられているとする。3D Gaussian Splatting（3DGS）では、三次元空間内のシーンを複数の 3D Gaussian（Gaussian）によって表現し、任意視点から観測される画像を再構成する。本稿では、3DGS の定式化について詳しく説明する。

2 文字の定義

本稿では、以下の表記規則を用いる。

- 小文字：スカラー
- 太字の小文字：ベクトル
- 大文字：行列

文字	定義
i	Gaussian のインデックス ($i = 1, \dots, N$)
$\mathbf{p}_{w,i}$	Gaussian の世界座標系における中心位置
$\mathbf{q}_i$	Gaussian の回転（四元数）
$\mathbf{s}_i$	Gaussian の各軸方向のスケール
$\mathbf{h}_i$	Gaussian の球面調和関数係数
α_i	Gaussian の不透明度
R_{cw}	世界座標系からカメラ座標系への回転行列
$\mathbf{t}_{cw}$	世界座標系からカメラ座標系への並進ベクトル
f_x, f_y	x, y 方向の焦点距離
c_x, c_y	画像上の主点座標
j	画素のインデックス ($j = 1, \dots, N_{\text{pix}}$)
γ_j^{gt}	真値画像の画素 j の RGB 値
γ_j	レンダリング画像の画素 j の RGB 値

表 1 本稿で使用する記号の定義

以下では、表記を簡潔にするため、特に必要な場合を除いて Gaussian を表す添字 i を省略する。

3 Gaussian の初期化

3DGS では、Structure from Motion (SfM) によって復元された疎な点群を用いて、3D Gaussian を初期化する。SfM によって得られた i 番目の 3D 点の位置を $\mathbf{p}_i^{\text{SfM}}$ とすると、Gaussian の中心位置は

$$\mathbf{p}_{w,i} = \mathbf{p}_i^{\text{SfM}} \quad (1)$$

と初期化される。

Gaussian の色は、SfM 点に対応する RGB 値を用いて初期化する。具体的には、球面調和関数の 0 次係数を RGB 値に基づいて設定し、それより高次の係数を 0 とする。¹⁾

また、Gaussian のスケールは、周囲の SfM 点までの距離に基づいて決定する。点群が密な領域では小さな Gaussian、疎な領域では大きな Gaussian として初期化する。回転は単位回転、不透明度は小さな一定値によって初期化する。²⁾

4 全体の流れ

3DGS の学習は、Gaussian のパラメータ最適化と Gaussian 数の適応的な調整を繰り返すことで行われる。学習の全体的な流れを以下に示す。

- SfM によって得られた疎な点群を用いて Gaussian を初期化する。
- 現在の Gaussian のパラメータを用いて、選択した学習視点の 2D 画像をレンダリングする。

- 実際に実装する際は、SfM 点の RGB 値 $\mathbf{c}_i^{\text{SfM}}$ に対して $\mathbf{h}_{i,00} = (\mathbf{c}_i^{\text{SfM}} - \frac{1}{2}\mathbf{1})/Y_0^0$ と初期化し、それより高次の球面調和係数を 0 とする。
- 実装では、SfM 点 i の近傍 3 点の集合を $\mathcal{K}_i$ とし、近傍点までの平均二乗距離を

$$d_i^2 = \frac{1}{|\mathcal{K}_i|} \sum_{k \in \mathcal{K}_i} \|\mathbf{p}_i^{\text{SfM}} - \mathbf{p}_k^{\text{SfM}}\|_2^2$$

とする。初期スケールとその raw パラメータは

$$\mathbf{s}_i = \sqrt{\max(d_i^2, 10^{-7})} \mathbf{1}, \quad \tilde{\mathbf{s}}_i = \log \mathbf{s}_i$$

とする。また、 $\tilde{\mathbf{q}}_i = [1, 0, 0, 0]^\top$ 、 $\alpha_i = 0.1$ 、 $\tilde{\alpha}_i = \text{logit}(0.1)$ と初期化する。ここで、 $\text{logit}(a) = \log\{a/(1-a)\}$ である。

3. レンダリング画像と真値画像を比較し、損失関数を計算する。
4. 損失関数を各 Gaussian のパラメータについて微分し、勾配を計算する。
5. 画面上の位置勾配や Gaussian の投影半径など、適応的密度制御に用いる統計量を蓄積する。
6. 所定の反復において、Gaussian の clone、split および prune による適応的密度制御を行う。また、必要に応じて不透明度をリセットする。
7. 得られた勾配を用いて、Gaussian の位置、回転、スケール、不透明度および球面調和係数を更新する。
8. 学習の進行に応じて、レンダリングに使用する球面調和関数の次数を段階的に増加させる。
9. 所定の反復回数に達するまで、手順 2 から 8 を繰り返す。

5 順伝播過程

本節では、Gaussian のパラメータを用いて、選択した学習視点の 2D 画像をレンダリングする処理を順伝播と呼ぶ。以下では、その定式化について説明する。

5.1 カメラ座標系における中心位置 p_c

世界座標系からカメラ座標系への回転行列 R_{cw} と並進ベクトル t_{cw} を用いて、Gaussian の世界座標における中心位置 p_w を、カメラ座標系における中心位置 p_c へ変換する。この座標変換は、式 (2) で表される。

$$\begin{aligned} p_c &= R_{cw}(p_w - c_w) \\ &= R_{cw}p_w - R_{cw}c_w \\ &= R_{cw}p_w + t_{cw} \end{aligned} \quad (2)$$

ここで、 c_w は世界座標系におけるカメラの中心位置であり、並進ベクトル t_{cw} との間には $t_{cw} = -R_{cw}c_w$ の関係がある。

5.2 p_c を画像平面上に投影した座標 u

カメラ座標系における Gaussian の中心位置を $p_c = [x, y, z]^\top$ とする。カメラ内部パラメータ f_x, f_y, c_x, c_y を用いると、 p_c を画像平面上に投影した座標 u は、式 (3) で表される。

$$u = \begin{bmatrix} f_x \frac{x}{z} + c_x \\ f_y \frac{y}{z} + c_y \end{bmatrix} \quad (3)$$

5.3 Gaussian の共分散行列 Σ

Gaussian の 3D 共分散行列 Σ は、行列の各成分を直接保持するのではなく、回転を表す単位クォータニオン q と、各軸方向のスケール s から構成する。この共分散行列は、式 (4) で表される。

$$\begin{aligned} \Sigma &= \Sigma(q, s) = RSS^\top R^\top, \\ \sigma &= \begin{bmatrix} \Sigma_{00} & \Sigma_{01} & \Sigma_{02} & \Sigma_{11} & \Sigma_{12} & \Sigma_{22} \end{bmatrix}^\top \end{aligned} \quad (4)$$

ここで、 R は単位クォータニオン $q = [q_w, q_x, q_y, q_z]^\top$ に対応する回転行列であり、式 (5) で表される。

$$R = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \quad (5)$$

また、 S はスケールベクトル $s = [s_0, s_1, s_2]^\top$ を対角成分に持つ行列であり、式 (6) で表される。

$$S = \begin{bmatrix} s_0 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_2 \end{bmatrix} \quad (6)$$

式 (4) は $\Sigma = AA^\top$ (ただし、 $A = RS$) という形であるため、 Σ は対称半正定値行列となる。また、 Σ は対称行列であるため、実装では重複しない上三角の 6 要素を中間表現としてベクトル σ に格納する。

5.4 画像上の 2D 共分散 Σ'

3D 共分散 Σ を画像平面上に投影した 2D 共分散 Σ' は、透視投影を p_c の周りで線形近似することで、式 (7) のように求められる。

$$\begin{aligned} \Sigma' &= JR_{cw}\Sigma R_{cw}^\top J^\top, \\ \sigma' &= \begin{bmatrix} \Sigma'_{00} & \Sigma'_{01} & \Sigma'_{11} \end{bmatrix}^\top \end{aligned} \quad (7)$$

ここで、 J は投影座標 u の $p_c = [x, y, z]^\top$ に関するヤコビ行列であり、式 (8) で表される。

$$J = \frac{\partial u}{\partial p_c} = \begin{bmatrix} \frac{f_x}{z} & 0 & -\frac{f_x x}{z^2} \\ 0 & \frac{f_y}{z} & -\frac{f_y y}{z^2} \end{bmatrix} \quad (8)$$

5.5 球面調和関数による色 c

3DGS では、視点によって変化する Gaussian の色を球面調和関数によって表現する。理論上、RGB 値 c は式 (9)

で計算される。³⁾

$$\mathbf{c}(\mathbf{r}, \mathbf{h}) = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \mathbf{h}_{lm} Y_l^m(\mathbf{r}) \quad (9)$$

ここで、 $\mathbf{r}$ はカメラ位置 $\mathbf{c}_w$ から Gaussian の中心位置 $\mathbf{p}_w$ へ向かう単位方向ベクトルであり、式 (11) で表される。

$$\mathbf{r} = \frac{\mathbf{p}_w - \mathbf{c}_w}{\|\mathbf{p}_w - \mathbf{c}_w\|} \quad (11)$$

$\mathbf{h}_{lm} \in \mathbb{R}^3$ は RGB 各成分に対応する球面調和係数、 $Y_l^m(\mathbf{r})$ は l 次 m 番目の実球面調和基底関数、 $l_{\max}$ は球面調和関数の最大次数である。

5.6 レンダリング画像の画素色 γ_j

これまでに得られた情報を用いて、レンダリング画像の画素 j の RGB 値 γ_j を計算する。画素 j に重なる Gaussian の集合を $\mathcal{N}_j$ とする。また、理論上の合成順序として、画素 j において Gaussian k が Gaussian i より手前にあることを $k \prec_j i$ と表し、背景色は黒とする。このとき、 γ_j は式 (12) で表される。

$$\gamma_j = \sum_{i \in \mathcal{N}_j} \tau_{ij} \alpha'_{ij} \mathbf{c}_i \quad (12)$$

ここで、 $\mathbf{c}_i$ は i 番目の Gaussian の RGB 値、 α'_{ij} は画素 j における i 番目の Gaussian の不透明度である。 α'_{ij} は、式 (13) で計算される。

$$\alpha'_{ij} = \alpha_i \exp \left(-\frac{1}{2} (\mathbf{x}_j - \mathbf{u}_i)^\top \Sigma_i'^{-1} (\mathbf{x}_j - \mathbf{u}_i) \right) \quad (13)$$

ここで、 α_i は Gaussian が保持する不透明度、 $\mathbf{u}_i$ は画像上に投影された Gaussian の中心、 $\mathbf{x}_j$ は画素 j の座標である。また、 Σ_i' は σ_i' から復元した 2D 共分散行列である。 $\mathbf{x}_j$ と $\mathbf{u}_i$ のマハラノビス距離が小さいほど、 α'_{ij} は大きくなる。

τ_{ij} は、画素 j において、 i 番目の Gaussian より手前にある Gaussian を透過した割合であり、式 (14) で計算される。

$$\tau_{ij} = \prod_{\substack{k \in \mathcal{N}_j \\ k \prec_j i}} (1 - \alpha'_{kj}) \quad (14)$$

3) 一方、実装では、球面調和関数の出力に 0.5 を加えた後、RGB 値の各成分を非負に制限する。この処理は式 (10) で表される。

$$\tilde{\mathbf{c}} = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \mathbf{h}_{lm} Y_l^m(\mathbf{r}), \quad \mathbf{c} = \max \left(\mathbf{0}, \tilde{\mathbf{c}} + \frac{1}{2} \mathbf{1} \right) \quad (10)$$

積を取る Gaussian が存在しない場合、空積の定義より最も手前にある Gaussian の透過率は 1 となる。したがって、手前の Gaussian の不透明度が大きいほど、奥にある Gaussian の色の寄与は小さくなる。⁴⁾ 2D 共分散行列を

$$\Sigma_i' = \begin{bmatrix} a_i & b_i \\ b_i & c_i \end{bmatrix} \quad (15)$$

とすると、その逆行列は式 (16) で表される。

$$\Sigma_i'^{-1} = \frac{1}{a_i c_i - b_i^2} \begin{bmatrix} c_i & -b_i \\ -b_i & a_i \end{bmatrix} \quad (16)$$

ここで、 a_i 、 b_i 、 c_i は 2D 共分散行列 Σ_i' の重複しない 3 成分である。

6 損失関数

順伝播によって得られたレンダリング画像と真値画像の差を評価するため、3DGS の学習に用いる全体の損失関数 $\mathcal{L}$ は、 L_1 損失と D-SSIM 損失の加重和として、式 (17) で表される。

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}} \quad (17)$$

ここで、 $\lambda \in [0, 1]$ は二つの損失の重みを調整するハイパーパラメータである。⁵⁾ λ を大きくすると画像の構造的な類似度が、小さくすると画素ごとの RGB 値の一致が重視される。

6.1 L_1 損失

L_1 損失は、レンダリング画像と真値画像における各画素の RGB 値の絶対誤差を平均したものである。レンダリング画像の画素 j の RGB 値を γ_j 、真値画像の画素 j の RGB 値を γ_j^{gt} とすると、 L_1 損失 $\mathcal{L}_1$ は式 (18) で表される。

$$\mathcal{L}_1 = \frac{1}{3N_{\text{pix}}} \sum_{j=1}^{N_{\text{pix}}} \|\gamma_j - \gamma_j^{\text{gt}}\|_1 \quad (18)$$

ここで、 N_{pix} は画像の総画素数であり、係数 3 は RGB の 3 チャンネルに対応する。また、RGB ベクトル

4) 実際の実装では、画面を 16×16 画素のタイルに分割し、各 Gaussian を投影範囲と重なるタイルに割り当てる。各 Gaussian はタイル ID とカメラ座標系における深度を用いてソートされ、各タイルを GPU 上の 1 つの thread block で処理する。各画素では、ソートされた Gaussian を手前から奥へ走査して α ブレンディングを行い、透過率が十分小さくなった時点で計算を終了する。なお、画素ごとの追加の深度ソートは行わないため、式 (12) における深度順の合成は実装上近似的に行われる。

5) 実際の実装する際は、 $\lambda = 0.2$ を用いる。

$\gamma = [\gamma_R, \gamma_G, \gamma_B]^\top$ の L_1 ノルムは、

$$\|\gamma\|_1 = |\gamma_R| + |\gamma_G| + |\gamma_B| \quad (19)$$

で定義される。したがって、 $\mathcal{L}_1$ が小さいほど、レンダリング画像の各画素の RGB 値が真値画像の RGB 値に近いことを表す。

6.2 D-SSIM 損失

L_1 損失は各画素の RGB 値を個別に比較するため、画像内の明るさ、コントラスト、構造といった局所的な特徴を直接評価することはできない。そこで、3DGS では、画像の局所的な構造の類似度を評価する SSIM を併用する。

レンダリング画像の画素 j を中心とする局所領域を P_j 、それに対応する真値画像の局所領域を P_j^{gt} とすると、両者の SSIM は式 (20) で表される。

$$\text{SSIM}(P_j, P_j^{\text{gt}}) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (20)$$

ここで、両局所領域 P_j, P_j^{gt} の要素をそれぞれ x_k, y_k とし、 μ_x および μ_y は各局所領域の加重平均、 σ_x^2 および σ_y^2 は加重分散、 σ_{xy} は加重共分散である。これらは、重みの総和が 1 となる局所窓の重み w_k ⁶⁾ を用いて、式 (21) で計算される。

$$\begin{aligned} \mu_x &= \sum_k w_k x_k, & \mu_y &= \sum_k w_k y_k, \\ \sigma_x^2 &= \sum_k w_k (x_k - \mu_x)^2, & \sigma_y^2 &= \sum_k w_k (y_k - \mu_y)^2, \\ \sigma_{xy} &= \sum_k w_k (x_k - \mu_x)(y_k - \mu_y) \end{aligned} \quad (21)$$

また、 C_1 および C_2 は、分母が極端に小さくなって計算が不安定になることを防ぐための定数であり、

$$C_1 = (K_1 L)^2, \quad C_2 = (K_2 L)^2 \quad (22)$$

と定義される。ここで、 L は画素値のダイナミックレンジであり、RGB 値を $[0, 1]$ に正規化する場合は $L = 1$ である。⁷⁾

画像全体の SSIM は、各画素および RGB 各チャンネルについて計算した SSIM の平均として求める。SSIM は値が大きいほど二つの画像が類似していることを表すため、損

6) 実際には実装する際は、大きさ 11×11 、標準偏差 1.5 の正規化された 2 次元ガウス窓を用いる。この窓を RGB 各チャンネルに対して独立に畳み込み、画像端には幅 5 のゼロパディングを適用する。得られた SSIM を全画素および RGB 全チャンネルについて平均し、画像全体の SSIM とする。

7) 実装する際は、 $K_1 = 0.01$ 、 $K_2 = 0.03$ を用いる。

失関数として最小化できるように、D-SSIM 損失を式 (23) で定義する。

$$\mathcal{L}_{\text{D-SSIM}} = 1 - \text{SSIM}(\gamma, \gamma^{\text{gt}}) \quad (23)$$

したがって、レンダリング画像と真値画像の構造が類似するほど $\mathcal{L}_{\text{D-SSIM}}$ は小さくなる。

7 逆伝播過程

本節では、レンダリング画像と真値画像から計算された損失を、各 Gaussian の学習パラメータへ逆向きに伝播させる過程を説明する。この過程を逆伝播過程と呼ぶ。

7.1 連鎖律とヤコビ行列

i 番目の Gaussian が保持する学習パラメータを

$$\theta_i = [p_{w,i}^\top \quad q_i^\top \quad s_i^\top \quad \alpha_i \quad h_i^\top]^\top \quad (24)$$

とする。すべての Gaussian の学習パラメータをまとめたものを θ とすると、3DGS の学習は、式 (25) の最適化問題として表される。

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\gamma(\theta), \gamma^{\text{gt}}) \quad (25)$$

ここで、 $\gamma(\theta)$ は学習パラメータ θ を用いてレンダリングされた画像、 γ^{gt} は真値画像である。

学習パラメータを更新するためには、各パラメータに関する損失関数の勾配を求める必要がある。損失関数 $\mathcal{L}$ はスカラーであるため、 θ_i に関する微分は勾配 $\nabla_{\theta_i} \mathcal{L}$ となる。この勾配は、各パラメータを微小に変化させたときに、損失がどの方向へどの程度変化するかを表す。

したがって、連鎖律により θ_i に関する損失の勾配はそれぞれ以下の式 (26)–(30) のようになる。

回転を表すクォータニオン q_i に関する勾配

$$\frac{\partial \mathcal{L}}{\partial q_i} = \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial \sigma'_i} \frac{\partial \sigma'_i}{\partial \sigma_i} \frac{\partial \sigma_i}{\partial q_i} \quad (26)$$

スケール s_i に関する勾配

$$\frac{\partial \mathcal{L}}{\partial s_i} = \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial \sigma'_i} \frac{\partial \sigma'_i}{\partial \sigma_i} \frac{\partial \sigma_i}{\partial s_i} \quad (27)$$

球面調和係数 h_i に関する勾配

$$\frac{\partial \mathcal{L}}{\partial h_i} = \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial c_i} \frac{\partial c_i}{\partial h_i} \quad (28)$$

不透明度 α_i に関する勾配

$$\frac{\partial \mathcal{L}}{\partial \alpha_i} = \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial \alpha_i} \quad (29)$$

中心位置 $p_{w,i}$ に関する勾配

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial p_{w,i}} &= \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial u_i} \frac{\partial u_i}{\partial p_{c,i}} \frac{\partial p_{c,i}}{\partial p_{w,i}} \\ &+ \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial c_i} \frac{\partial c_i}{\partial r_i} \frac{\partial r_i}{\partial p_{w,i}} \\ &+ \sum_j \frac{\partial \mathcal{L}}{\partial \gamma_j} \frac{\partial \gamma_j}{\partial \sigma'_i} \frac{\partial \sigma'_i}{\partial p_{c,i}} \frac{\partial p_{c,i}}{\partial p_{w,i}} \end{aligned} \quad (30)$$

なお、損失関数の画素色 γ_j に関する偏導関数 $\partial \mathcal{L} / \partial \gamma_j$ の導出は本稿では省略する。また、本節で示す偏導関数は、3DGS における逆伝播を数式的に表したものである。⁸⁾

7.2 座標変換および投影の偏導関数

本小節では、カメラ座標 $p_{c,i}$ および投影中心 u_i をそれぞれ $p_{w,i}$, $p_{c,i}$ について微分し、以下のヤコビ行列を導出する。

$$\frac{\partial p_{c,i}}{\partial p_{w,i}}, \quad \frac{\partial u_i}{\partial p_{c,i}}$$

式 (2) より、世界座標系からカメラ座標系への変換は

$$p_{c,i} = R_{cw} p_{w,i} + t_{cw} \quad (31)$$

である。 R_{cw} および t_{cw} は固定されたカメラパラメータであるため、

$$\frac{\partial p_{c,i}}{\partial p_{w,i}} = R_{cw} \quad (32)$$

となる。

次に、 $p_{c,i} = [x_i, y_i, z_i]^\top$ とすると、式 (3) より、投影中心は

$$u_i = \begin{bmatrix} f_x x_i / z_i + c_x \\ f_y y_i / z_i + c_y \end{bmatrix} \quad (33)$$

と表される。したがって、 u_i の $p_{c,i}$ に関するヤコビ行列は

$$\frac{\partial u_i}{\partial p_{c,i}} = J_i = \begin{bmatrix} \frac{f_x}{z_i} & 0 & -\frac{f_x x_i}{z_i^2} \\ 0 & \frac{f_y}{z_i} & -\frac{f_y y_i}{z_i^2} \end{bmatrix} \quad (34)$$

となる。

8) 実際の実装では、損失関数からレンダリング画像までの勾配計算は PyTorch の自動微分によって行われる。Gaussian のラスタライズ処理には公式実装の CUDA 拡張 `diff-gaussian-rasterization` を使用し、その custom backward によって Gaussian の各パラメータに関する勾配を計算する。また、適応的密度制御では、画像平面上に投影された Gaussian 中心の勾配も取得し、Gaussian の clone および split の判定に用いる。

7.3 3D 共分散の偏導関数

本小節では、3D 共分散 σ_i を q_i , s_i について微分し、以下のヤコビ行列を導出する。

$$\frac{\partial \sigma_i}{\partial q_i}, \quad \frac{\partial \sigma_i}{\partial s_i}$$

式 (4) より、3D 共分散行列は

$$\Sigma_i = R_i D_i R_i^\top, \quad D_i = S_i S_i^\top = \text{diag}(s_{i,0}^2, s_{i,1}^2, s_{i,2}^2) \quad (35)$$

と表される。

クォータニオンの成分を $q_{i,k}$ ($k \in \{w, x, y, z\}$) とする。 D_i は q_i に依存しないため、積の微分法より

$$\frac{\partial \Sigma_i}{\partial q_{i,k}} = \frac{\partial R_i}{\partial q_{i,k}} D_i R_i^\top + R_i D_i \left(\frac{\partial R_i}{\partial q_{i,k}} \right)^\top \quad (36)$$

となる。

ここで、式 (5) を各クォータニオン成分について偏微分すると、

$$\frac{\partial R_i}{\partial q_{i,w}} = 2 \begin{bmatrix} 0 & -q_{i,z} & q_{i,y} \\ q_{i,z} & 0 & -q_{i,x} \\ -q_{i,y} & q_{i,x} & 0 \end{bmatrix} \quad (37)$$

$$\frac{\partial R_i}{\partial q_{i,x}} = 2 \begin{bmatrix} 0 & q_{i,y} & q_{i,z} \\ q_{i,y} & -2q_{i,x} & -q_{i,w} \\ q_{i,z} & q_{i,w} & -2q_{i,x} \end{bmatrix} \quad (38)$$

$$\frac{\partial R_i}{\partial q_{i,y}} = 2 \begin{bmatrix} -2q_{i,y} & q_{i,x} & q_{i,w} \\ q_{i,x} & 0 & q_{i,z} \\ -q_{i,w} & q_{i,z} & -2q_{i,y} \end{bmatrix} \quad (39)$$

$$\frac{\partial R_i}{\partial q_{i,z}} = 2 \begin{bmatrix} -2q_{i,z} & -q_{i,w} & q_{i,x} \\ q_{i,w} & -2q_{i,z} & q_{i,y} \\ q_{i,x} & q_{i,y} & 0 \end{bmatrix} \quad (40)$$

を得る。

各 $q_{i,k}$ について式 (36) を計算し、得られた行列の上三角 6 要素を式 (4) における σ_i と同じ順序で並べることで、 $\partial \sigma_i / \partial q_{i,k}$ が得られる。したがって、求めるヤコビ行列は

$$\frac{\partial \sigma_i}{\partial q_i} = \begin{bmatrix} \frac{\partial \sigma_i}{\partial q_{i,w}} & \frac{\partial \sigma_i}{\partial q_{i,x}} & \frac{\partial \sigma_i}{\partial q_{i,y}} & \frac{\partial \sigma_i}{\partial q_{i,z}} \end{bmatrix} \quad (41)$$

となる。

次に、 e_k を第 k 成分のみが 1 である 3 次元標準基底ベクトルとする。このとき、

$$\frac{\partial D_i}{\partial s_{i,k}} = 2s_{i,k} e_k e_k^\top, \quad k \in \{0, 1, 2\} \quad (42)$$

である。したがって、

$$\frac{\partial \Sigma_i}{\partial s_{i,k}} = 2s_{i,k} R_i \mathbf{e}_k \mathbf{e}_k^\top R_i^\top \quad (43)$$

となる。

各 $s_{i,k}$ について式 (43) を計算し、得られた行列の上三角 6 要素を式 (4) における σ_i と同じ順序で並べることで、 $\partial \sigma_i / \partial s_{i,k}$ が得られる。したがって、求めるヤコビ行列は

$$\frac{\partial \sigma_i}{\partial \mathbf{s}_i} = \begin{bmatrix} \frac{\partial \sigma_i}{\partial s_{i,0}} & \frac{\partial \sigma_i}{\partial s_{i,1}} & \frac{\partial \sigma_i}{\partial s_{i,2}} \end{bmatrix} \quad (44)$$

となる。

7.4 2D 共分散の偏導関数

本小節では、2D 共分散 σ'_i を $\sigma_i, \mathbf{p}_{c,i}$ について微分し、以下のヤコビ行列を導出する。

$$\frac{\partial \sigma'_i}{\partial \sigma_i}, \quad \frac{\partial \sigma'_i}{\partial \mathbf{p}_{c,i}}$$

式 (7) に対して、

$$C_i = R_{cw} \Sigma_i R_{cw}^\top \quad (45)$$

とおくと、2D 共分散行列は

$$\Sigma'_i = J_i C_i J_i^\top \quad (46)$$

と表される。

まず、 σ_i の第 r 成分について考える。式 (4) より、 Σ_i は σ_i の各成分を対応する上三角要素および下三角要素に配置した対称行列である。したがって、

$$\frac{\partial \Sigma'_i}{\partial [\sigma_i]_r} = J_i R_{cw} \frac{\partial \Sigma_i}{\partial [\sigma_i]_r} R_{cw}^\top J_i^\top, \quad r \in \{0, \dots, 5\} \quad (47)$$

となる。

各 r について式 (47) を計算し、得られた行列の上三角 3 要素を式 (7) における σ'_i と同じ順序で並べることで、 $\partial \sigma'_i / \partial [\sigma_i]_r$ が得られる。したがって、求めるヤコビ行列の第 r 列は

$$\left[\frac{\partial \sigma'_i}{\partial \sigma_i} \right]_{:,r} = \frac{\partial \sigma'_i}{\partial [\sigma_i]_r}, \quad r \in \{0, \dots, 5\} \quad (48)$$

となる。

次に、 $\mathbf{p}_{c,i} = [x_i, y_i, z_i]^\top$ の第 k 成分に関する J_i の偏導関数を $J_{i,k}$ とする。式 (34) より、

$$J_{i,0} = \frac{\partial J_i}{\partial x_i} = \begin{bmatrix} 0 & 0 & -f_x/z_i^2 \\ 0 & 0 & 0 \end{bmatrix} \quad (49)$$

$$J_{i,1} = \frac{\partial J_i}{\partial y_i} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -f_y/z_i^2 \end{bmatrix} \quad (50)$$

および

$$J_{i,2} = \frac{\partial J_i}{\partial z_i} = \begin{bmatrix} -f_x/z_i^2 & 0 & 2f_x x_i/z_i^3 \\ 0 & -f_y/z_i^2 & 2f_y y_i/z_i^3 \end{bmatrix} \quad (51)$$

を得る。

C_i は $\mathbf{p}_{c,i}$ に依存しないため、積の微分法より

$$\frac{\partial \Sigma'_i}{\partial [\mathbf{p}_{c,i}]_k} = J_{i,k} C_i J_i^\top + J_i C_i J_{i,k}^\top, \quad k \in \{0, 1, 2\} \quad (52)$$

となる。

各 k について式 (52) を計算し、得られた行列の上三角 3 要素を式 (7) における σ'_i と同じ順序で並べることで、 $\partial \sigma'_i / \partial [\mathbf{p}_{c,i}]_k$ が得られる。したがって、求めるヤコビ行列の第 k 列は

$$\left[\frac{\partial \sigma'_i}{\partial \mathbf{p}_{c,i}} \right]_{:,k} = \frac{\partial \sigma'_i}{\partial [\mathbf{p}_{c,i}]_k}, \quad k \in \{0, 1, 2\} \quad (53)$$

となる。

7.5 色の偏導関数

本小節では、Gaussian の色 $\mathbf{c}_i$ を $\mathbf{h}_i, \mathbf{p}_{w,i}$ について微分し、以下のヤコビ行列を導出する。

$$\frac{\partial \mathbf{c}_i}{\partial \mathbf{h}_i}, \quad \frac{\partial \mathbf{c}_i}{\partial \mathbf{p}_{w,i}}$$

式 (9) より、Gaussian i の色は

$$\mathbf{c}_i = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \mathbf{h}_{i,lm} Y_l^m(\mathbf{r}_i) \quad (54)$$

と表される。

まず、 $\mathbf{c}_i$ を球面調和係数 $\mathbf{h}_{i,lm}$ について偏微分すると、

$$\frac{\partial \mathbf{c}_i}{\partial \mathbf{h}_{i,lm}} = Y_l^m(\mathbf{r}_i) \mathbf{I}_3 \quad (55)$$

となる。⁹⁾

$\mathbf{h}_i$ を、各球面調和係数 $\mathbf{h}_{i,lm} \in \mathbb{R}^3$ を一つのベクトルに並べたものとする、求めるヤコビ行列は

$$\frac{\partial \mathbf{c}_i}{\partial \mathbf{h}_i} = \begin{bmatrix} Y_0^0(\mathbf{r}_i) \mathbf{I}_3 & \cdots & Y_l^m(\mathbf{r}_i) \mathbf{I}_3 & \cdots & Y_{l_{\max}}^{l_{\max}}(\mathbf{r}_i) \mathbf{I}_3 \end{bmatrix} \quad (56)$$

となる。ただし、各ブロックは、 $\mathbf{h}_i$ における球面調和係数の並び順に配置する。

9) 式 (10) の実装式を用いる場合、非負制約によって 0 に切り詰められた RGB 成分の勾配も 0 となる。

次に、 $\mathbf{c}_i$ の $\mathbf{p}_{w,i}$ に関するヤコビ行列を求める。式 (11) より、

$$\mathbf{r}_i = \frac{\mathbf{p}_{w,i} - \mathbf{c}_w}{\|\mathbf{p}_{w,i} - \mathbf{c}_w\|_2} \quad (57)$$

である。単位ベクトルの微分より、

$$\frac{\partial \mathbf{r}_i}{\partial \mathbf{p}_{w,i}} = \frac{I_3 - \mathbf{r}_i \mathbf{r}_i^\top}{\|\mathbf{p}_{w,i} - \mathbf{c}_w\|_2} \quad (58)$$

となる。

また、式 (54) を $\mathbf{r}_i$ について偏微分すると、

$$\frac{\partial \mathbf{c}_i}{\partial \mathbf{r}_i} = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l \mathbf{h}_{i,lm} \{\nabla_{\mathbf{r}_i} Y_l^m(\mathbf{r}_i)\}^\top \quad (59)$$

となる。

したがって、連鎖律より、 $\mathbf{c}_i$ の $\mathbf{p}_{w,i}$ に関するヤコビ行列は

$$\begin{aligned} \frac{\partial \mathbf{c}_i}{\partial \mathbf{p}_{w,i}} &= \frac{\partial \mathbf{c}_i}{\partial \mathbf{r}_i} \frac{\partial \mathbf{r}_i}{\partial \mathbf{p}_{w,i}} \\ &= \frac{1}{\|\mathbf{p}_{w,i} - \mathbf{c}_w\|_2} \frac{\partial \mathbf{c}_i}{\partial \mathbf{r}_i} (I_3 - \mathbf{r}_i \mathbf{r}_i^\top) \end{aligned} \quad (60)$$

となる。

7.6 画素色の偏導関数

本小節では、画素色 γ_j を α_i , $\mathbf{c}_i$, $\mathbf{u}_i$, σ'_i について微分し、以下のヤコビ行列を導出する。

$$\frac{\partial \gamma_j}{\partial \alpha_i}, \quad \frac{\partial \gamma_j}{\partial \mathbf{c}_i}, \quad \frac{\partial \gamma_j}{\partial \mathbf{u}_i}, \quad \frac{\partial \gamma_j}{\partial \sigma'_i}$$

画素色の計算式を以下に示す。

$$\gamma_j = \sum_{i \in \mathcal{N}_j} \tau_{ij} \alpha'_i \mathbf{c}_i \quad (61)$$

$$\tau_{ij} = \prod_{\substack{k \in \mathcal{N}_j \\ k \prec j}} (1 - \alpha'_{kj}) \quad (62)$$

まず、 α'_i に関する偏導関数を求める。 α'_i は Gaussian i 自身の色だけでなく、それより奥にある Gaussian の透過率にも影響するため、式 (61) を直接微分するのは煩雑である。そこで、本小節に限り、画素 j に重なる Gaussian をカメラから近い順に $1, \dots, M_j$ と付け直し、画素色の合成式を以下の再帰形式に書き直す。

$$\begin{aligned} \gamma_{M_j+1,j} &= \mathbf{0}, \\ \gamma_{M_j,j} &= \alpha'_{M_j,j} \mathbf{c}_{M_j} + (1 - \alpha'_{M_j,j}) \gamma_{M_j+1,j}, \\ &\vdots \\ \gamma_{2,j} &= \alpha'_{2,j} \mathbf{c}_2 + (1 - \alpha'_{2,j}) \gamma_{3,j}, \\ \gamma_{1,j} &= \alpha'_{1,j} \mathbf{c}_1 + (1 - \alpha'_{1,j}) \gamma_{2,j}, \\ \gamma_j &= \gamma_{1,j} \end{aligned} \quad (63)$$

ここで、 $\gamma_{i,j}$ は、深度順で i 番目から M_j 番目までの Gaussian のみを合成した画素 j の色を表す。¹⁰⁾

式 (63) より、 $\gamma_{i,j}$ の α'_{ij} に関する偏導関数は

$$\frac{\partial \gamma_{i,j}}{\partial \alpha'_{ij}} = \mathbf{c}_i - \gamma_{i+1,j} \quad (64)$$

となる。

また、 γ_j の $\gamma_{i,j}$ に関するヤコビ行列は

$$\begin{aligned} \frac{\partial \gamma_j}{\partial \gamma_{i,j}} &= \prod_{k=1}^{i-1} (1 - \alpha'_{kj}) I_3 \\ &= \tau_{ij} I_3 \end{aligned} \quad (65)$$

となる。したがって、連鎖律より、 γ_j の α'_{ij} に関する偏導関数は

$$\frac{\partial \gamma_j}{\partial \alpha'_{ij}} = \tau_{ij} (\mathbf{c}_i - \gamma_{i+1,j}) \quad (66)$$

となる。

一方、 $\mathbf{c}_i$ は Gaussian i 自身の項にのみ現れるため、 γ_j の $\mathbf{c}_i$ に関するヤコビ行列は

$$\frac{\partial \gamma_j}{\partial \mathbf{c}_i} = \tau_{ij} \alpha'_i I_3 \quad (67)$$

となる。

次に、 α'_i を α_i , $\mathbf{u}_i$ および σ'_i について微分する。式 (13) より、

$$\alpha'_i = \alpha_i \exp \left(-\frac{1}{2} (\mathbf{u}_i - \mathbf{x}_j)^\top \Sigma_i'^{-1} (\mathbf{u}_i - \mathbf{x}_j) \right) \quad (68)$$

$$\equiv \alpha_i g_{ij}$$

と表される。

2D 共分散行列とその逆行列を

$$\Sigma_i' = \begin{bmatrix} a_i & b_i \\ b_i & c_i \end{bmatrix}, \quad \Sigma_i'^{-1} = \begin{bmatrix} \omega'_{i,0} & \omega'_{i,1} \\ \omega'_{i,1} & \omega'_{i,2} \end{bmatrix} \quad (69)$$

とする。また、それぞれの重複しない上三角成分を

$$\sigma'_i = \begin{bmatrix} a_i & b_i & c_i \end{bmatrix}^\top, \quad \omega'_i = \begin{bmatrix} \omega'_{i,0} & \omega'_{i,1} & \omega'_{i,2} \end{bmatrix}^\top \quad (70)$$

とする。

さらに、

$$\mathbf{d}_{ij} = \mathbf{u}_i - \mathbf{x}_j = \begin{bmatrix} d_{ij,0} & d_{ij,1} \end{bmatrix}^\top \quad (71)$$

10) 実際の CUDA 実装では、順伝播時に用いたタイル単位の Gaussian 列を逆向きに走査することで逆伝播を行う。そのため、各画素について Gaussian 列を新たに構築して式 (63) を直接計算するわけではない。本小節の式は、その逆伝播によって計算される勾配を数式的に表したものである。

とおくと、 g_{ij} は

$$g_{ij} = \exp \left(-\frac{1}{2} \mathbf{d}_{ij}^\top \Sigma_i'^{-1} \mathbf{d}_{ij} \right) \\ = \exp \left(-\frac{1}{2} \omega'_{i,0} d_{ij,0}^2 - \omega'_{i,1} d_{ij,0} d_{ij,1} - \frac{1}{2} \omega'_{i,2} d_{ij,1}^2 \right) \quad (72)$$

と表される。

式 (68) より、 α'_{ij} の α_i に関する偏導関数は

$$\frac{\partial \alpha'_{ij}}{\partial \alpha_i} = g_{ij} \quad (73)$$

となる。

また、 g_{ij} を $\mathbf{u}_i$ について偏微分すると、

$$\frac{\partial g_{ij}}{\partial \mathbf{u}_i} = \begin{bmatrix} -\omega'_{i,0} d_{ij,0} & -\omega'_{i,1} d_{ij,1} & -\omega'_{i,1} d_{ij,0} - \omega'_{i,2} d_{ij,1} \end{bmatrix} g_{ij} \quad (74)$$

を得る。したがって、

$$\frac{\partial \alpha'_{ij}}{\partial \mathbf{u}_i} = \alpha_i \frac{\partial g_{ij}}{\partial \mathbf{u}_i} \quad (75)$$

となる。

次に、 g_{ij} を ω'_i について偏微分すると、

$$\frac{\partial g_{ij}}{\partial \omega'_i} = \begin{bmatrix} -\frac{1}{2} d_{ij,0}^2 & -d_{ij,0} d_{ij,1} & -\frac{1}{2} d_{ij,1}^2 \end{bmatrix} g_{ij} \quad (76)$$

となる。

ここで、

$$\Delta_i = a_i c_i - b_i^2 \quad (77)$$

とおくと、式 (16) より、

$$\omega'_i = \frac{1}{\Delta_i} \begin{bmatrix} c_i & -b_i & a_i \end{bmatrix}^\top \quad (78)$$

である。したがって、 ω'_i の σ'_i に関するヤコビ行列は

$$\frac{\partial \omega'_i}{\partial \sigma'_i} = \frac{1}{\Delta_i^2} \begin{bmatrix} -c_i^2 & 2b_i c_i & -b_i^2 \\ b_i c_i & -(a_i c_i + b_i^2) & a_i b_i \\ -b_i^2 & 2a_i b_i & -a_i^2 \end{bmatrix} \quad (79)$$

となる。よって、連鎖律より、

$$\frac{\partial \alpha'_{ij}}{\partial \sigma'_i} = \alpha_i \frac{\partial g_{ij}}{\partial \omega'_i} \frac{\partial \omega'_i}{\partial \sigma'_i} \quad (80)$$

を得る。

以上の結果を連鎖律によって合成する。まず、 γ_j の α_i に関する偏導関数は

$$\frac{\partial \gamma_j}{\partial \alpha_i} = \frac{\partial \gamma_j}{\partial \alpha'_{ij}} \frac{\partial \alpha'_{ij}}{\partial \alpha_i} \\ = \tau_{ij} (\mathbf{c}_i - \gamma_{i+1,j}) g_{ij} \quad (81)$$

となる。

次に、 γ_j の $\mathbf{u}_i$ に関するヤコビ行列は

$$\frac{\partial \gamma_j}{\partial \mathbf{u}_i} = \frac{\partial \gamma_j}{\partial \alpha'_{ij}} \frac{\partial \alpha'_{ij}}{\partial \mathbf{u}_i} \\ = \tau_{ij} (\mathbf{c}_i - \gamma_{i+1,j}) \alpha_i \frac{\partial g_{ij}}{\partial \mathbf{u}_i} \quad (82)$$

となる。

同様に、 γ_j の σ'_i に関するヤコビ行列は

$$\frac{\partial \gamma_j}{\partial \sigma'_i} = \frac{\partial \gamma_j}{\partial \alpha'_{ij}} \frac{\partial \alpha'_{ij}}{\partial \sigma'_i} \\ = \tau_{ij} (\mathbf{c}_i - \gamma_{i+1,j}) \alpha_i \frac{\partial g_{ij}}{\partial \omega'_i} \frac{\partial \omega'_i}{\partial \sigma'_i} \quad (83)$$

となる。

以上より、画素色 γ_j に関する偏導関数が得られた。

なお、 $i \notin \mathcal{N}_j$ の場合、以上の偏導関数はいずれも $\mathbf{0}$ となる。

8 パラメータの更新

逆伝播によって求めた勾配を用いて、各 Gaussian のパラメータを更新する。学習対象となるパラメータ θ_i は、式 (24) で定義したものとする。¹¹⁾ 各パラメータは、損失関数 $\mathcal{L}$ が小さくなる方向へ反復的に更新される。勾配降下法による基本的な更新式は、

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \eta \left(\frac{\partial \mathcal{L}}{\partial \theta_i} \right)^\top \quad (84)$$

である。¹²⁾ ここで、 t は更新回数、 η は学習率である。実際の学習では、各パラメータに異なる学習率を設定し、Adam を用いて更新する。¹³⁾

9 評価指標

評価指標には PSNR (Peak Signal-to-Noise Ratio)、SSIM (Structural Similarity Index Measure)、および LPIPS (Learned Perceptual Image Patch Similarity) を用いる。

11) 実装時に学習する raw パラメータとレンダリング時の変換については、小節 B.2 に示す。

12) 本稿では、 $\partial \mathcal{L} / \partial \theta_i$ を θ_i に関する偏導関数を並べた行ベクトルとして扱う。そのため、列ベクトルである θ_i を更新する際には、その転置を用いる。

13) 適応的の密度制御によって Gaussian の clone、split および prune を行う場合、Gaussian の総数が変化するため、学習パラメータを保持するテンソルの大きさも変化する。実装では、Gaussian の追加および削除に合わせて、Adam が各パラメータについて保持する optimizer state も更新する。

9.1 PSNR

PSNR は、レンダリング画像と真値画像の画素値誤差に基づいて画質を評価する指標である。

n 番目のテスト画像における MSE (Mean Squared Error) を

$$\text{MSE}_n = \frac{1}{3N_{\text{pix}}} \sum_{j=1}^{N_{\text{pix}}} \|\gamma_{n,j} - \gamma_{n,j}^{\text{gt}}\|_2^2 \quad (85)$$

と定義する。ここで、 N_{pix} は画像の総画素数、 $\gamma_{n,j}$ は n 番目のレンダリング画像における画素 j の RGB 値、 $\gamma_{n,j}^{\text{gt}}$ は対応する真値画像の RGB 値である。

このとき、 n 番目のテスト画像の PSNR は

$$\text{PSNR}_n = 10 \log_{10} \left(\frac{I_{\text{max}}^2}{\text{MSE}_n} \right) \quad (86)$$

と定義される。ここで、 I_{max} は画素値の最大値である。RGB 値のダイナミックレンジを $[0, 1]$ とする¹⁴⁾ため、 $I_{\text{max}} = 1$ である。PSNR の単位は dB であり、値が大きいほど、レンダリング画像と真値画像の画素値誤差が小さいことを表す。

9.2 SSIM

SSIM は、画像の輝度、コントラストおよび構造の類似度に基づいて画質を評価する指標である。SSIM の定義は、損失関数の節で示した式 (20) と同様である。SSIM は値が大きいほど、レンダリング画像と真値画像の構造的な類似度が高いことを表す。

9.3 LPIPS

LPIPS は、学習済みニューラルネットワークによって抽出された特徴量を比較することで、レンダリング画像と真値画像の知覚的な類似度を評価する指標である。画素値を直接比較する PSNR とは異なり、画像の見た目や高次の特徴の違いを評価する。

n 番目のテスト画像について、LPIPS を

$$\text{LPIPS}_n = d_{\text{LPIPS}}(\gamma_n, \gamma_n^{\text{gt}}) \quad (87)$$

と表す。ここで、 d_{LPIPS} は学習済みニューラルネットワークの特徴量に基づく知覚距離を表す。LPIPS は値が小さいほど、レンダリング画像と真値画像が知覚的に類似していることを表す。

14) 評価指標を計算する際は、レンダリング画像の各 RGB 値を $[0, 1]$ にクリップする。ただし、学習時の損失関数はクリップ前のレンダリング画像から計算する。

9.4 テスト画像全体の評価

各評価指標は各テスト画像について計算し、それらの算術平均をモデルの評価値とする。テスト画像の枚数を N_{test} とすると、平均 PSNR は

$$\overline{\text{PSNR}} = \frac{1}{N_{\text{test}}} \sum_{n=1}^{N_{\text{test}}} \text{PSNR}_n \quad (88)$$

と定義される。

同様に、平均 SSIM および平均 LPIPS を

$$\overline{\text{SSIM}} = \frac{1}{N_{\text{test}}} \sum_{n=1}^{N_{\text{test}}} \text{SSIM}_n \quad (89)$$

$$\overline{\text{LPIPS}} = \frac{1}{N_{\text{test}}} \sum_{n=1}^{N_{\text{test}}} \text{LPIPS}_n \quad (90)$$

と定義する。

A 予備知識

A.1 実球面調和関数

本稿では、以下の実球面調和基底関数を使用する。単位方向ベクトルを

$$\mathbf{r} = \begin{bmatrix} x & y & z \end{bmatrix}^T, \quad x^2 + y^2 + z^2 = 1 \quad (91)$$

とする。

以下に $l = 0, \dots, 3$ における実球面調和基底関数を示す。

$l = 0$

$$Y_0^0(\mathbf{r}) = \frac{1}{2} \sqrt{\frac{1}{\pi}} \quad (92)$$

$l = 1$

$$\begin{aligned} Y_1^{-1}(\mathbf{r}) &= -\sqrt{\frac{3}{4\pi}} y, \\ Y_1^0(\mathbf{r}) &= \sqrt{\frac{3}{4\pi}} z, \\ Y_1^1(\mathbf{r}) &= -\sqrt{\frac{3}{4\pi}} x \end{aligned} \quad (93)$$

$l = 2$

$$\begin{aligned} Y_2^{-2}(\mathbf{r}) &= \frac{1}{2} \sqrt{\frac{15}{\pi}} xy, \\ Y_2^{-1}(\mathbf{r}) &= -\frac{1}{2} \sqrt{\frac{15}{\pi}} yz, \\ Y_2^0(\mathbf{r}) &= \frac{1}{4} \sqrt{\frac{5}{\pi}} (3z^2 - 1), \\ Y_2^1(\mathbf{r}) &= -\frac{1}{2} \sqrt{\frac{15}{\pi}} xz, \\ Y_2^2(\mathbf{r}) &= \frac{1}{4} \sqrt{\frac{15}{\pi}} (x^2 - y^2) \end{aligned} \quad (94)$$

$l = 3$

$$\begin{aligned} Y_3^{-3}(\mathbf{r}) &= -\frac{1}{4} \sqrt{\frac{35}{2\pi}} y (3x^2 - y^2), \\ Y_3^{-2}(\mathbf{r}) &= \frac{1}{2} \sqrt{\frac{105}{\pi}} xyz, \\ Y_3^{-1}(\mathbf{r}) &= -\frac{1}{4} \sqrt{\frac{21}{2\pi}} y (5z^2 - 1), \\ Y_3^0(\mathbf{r}) &= \frac{1}{4} \sqrt{\frac{7}{\pi}} z (5z^2 - 3), \\ Y_3^1(\mathbf{r}) &= -\frac{1}{4} \sqrt{\frac{21}{2\pi}} x (5z^2 - 1), \\ Y_3^2(\mathbf{r}) &= \frac{1}{4} \sqrt{\frac{105}{\pi}} z (x^2 - y^2), \\ Y_3^3(\mathbf{r}) &= -\frac{1}{4} \sqrt{\frac{35}{2\pi}} x (x^2 - 3y^2) \end{aligned} \quad (95)$$

B 実装仕様

B.1 実装方針

本実装は Python および PyTorch を用いて行う。Gaussian の中心位置、スケール、不透明度、クォータニオンおよび球面調和係数を学習対象とし、スケール、不透明度およびクォータニオンについては raw パラメータを `torch.nn.Parameter` として保持する。

レンダリングには、PyTorch のテンソル演算によって構成した参照実装と、3DGS の公式実装で用いられている CUDA rasterizer を利用する CUDA 実装の 2 種類を用意する。参照実装は数式および勾配計算の検証に使用し、原論文の再現実験では CUDA 実装を使用する。¹⁵⁾

また、本実装では、学習中に Gaussian の clone、split および prune を行う適応的密度制御を導入する。このため、

Gaussian の総数は学習中に動的に変化する。さらに、不透明度のリセット、球面調和関数の次数の段階的な増加、および学習初期の解像度の段階的な増加を行う。

B.2 学習パラメータと変換

理論式では、Gaussian i のパラメータを

$$\theta_i = \begin{bmatrix} \mathbf{p}_{w,i}^\top & \mathbf{q}_i^\top & \mathbf{s}_i^\top & \alpha_i & \mathbf{h}_i^\top \end{bmatrix}^\top \quad (96)$$

と定義した。

一方、実装では、スケールおよび不透明度の有効範囲を保証し、クォータニオンを単位クォータニオンに制約するため、それぞれの raw パラメータ $\tilde{\mathbf{s}}_i \in \mathbb{R}^3$ 、 $\tilde{\alpha}_i \in \mathbb{R}$ および $\tilde{\mathbf{q}}_i \in \mathbb{R}^4$ を学習する。したがって、実装上の学習パラメータを

$$\tilde{\theta}_i = \begin{bmatrix} \mathbf{p}_{w,i}^\top & \tilde{\mathbf{q}}_i^\top & \tilde{\mathbf{s}}_i^\top & \tilde{\alpha}_i & \mathbf{h}_i^\top \end{bmatrix}^\top \quad (97)$$

と定義する。

レンダリングに使用するスケール、不透明度およびクォータニオンは、raw パラメータから

$$\mathbf{s}_i = \exp(\tilde{\mathbf{s}}_i), \quad \alpha_i = \text{sigmoid}(\tilde{\alpha}_i), \quad \mathbf{q}_i = \frac{\tilde{\mathbf{q}}_i}{\max(\|\tilde{\mathbf{q}}_i\|_2, \varepsilon_q)} \quad (98)$$

によって計算する。ここで、指数関数は $\tilde{\mathbf{s}}_i$ の各成分に適用し、 ε_q はゼロ除算を防ぐための定数である。本実装では、 $\varepsilon_q = 10^{-8}$ とする。

中心位置 $\mathbf{p}_{w,i}$ および球面調和係数 $\mathbf{h}_i$ には変換を適用せず、その値をそのままレンダリングに使用する。実装では、 $\mathbf{p}_{w,i}$ 、 $\tilde{\mathbf{q}}_i$ 、 $\tilde{\mathbf{s}}_i$ 、 $\tilde{\alpha}_i$ および $\mathbf{h}_i$ を `torch.nn.Parameter` として保持する。

raw パラメータからレンダリング用パラメータへの変換は PyTorch のテンソル演算として実装し、これらの変換に関する勾配は PyTorch の自動微分によって計算する。

B.3 Gaussian の初期化

SfM によって得られた点群を

$$\mathcal{P}_{\text{SfM}} = \{(\mathbf{p}_i^{\text{SfM}}, \mathbf{c}_i^{\text{SfM}})\}_{i=1}^N \quad (99)$$

とする。¹⁶⁾ ここで、 $\mathbf{p}_i^{\text{SfM}} \in \mathbb{R}^3$ は SfM 点の位置、 $\mathbf{c}_i^{\text{SfM}} \in [0, 1]^3$ は対応する RGB 値である。SfM 点 1 点につき一つの Gaussian を生成するため、初期 Gaussian 数は SfM 点数 N と等しい。

¹⁵⁾ CUDA 実装では、公式実装の `diff-gaussian-rasterization` を使用する。ラスタライズ処理の逆伝播には、同 CUDA 拡張に実装された custom backward が用いられる。

¹⁶⁾ Synthetic NeRF を用いた再現実験では SfM 点群を用いないため、初期 Gaussian をシーン内にランダムに配置して学習を開始する。

中心位置の初期化

Gaussian i の中心位置は、対応する SfM 点の位置を用いて

$$\mathbf{p}_{w,i} = \mathbf{p}_i^{\text{SfM}} \quad (100)$$

と初期化する。

スケールの初期化

Gaussian i に対応する SfM 点から最も近い 3 点のインデックス集合を $\mathcal{K}_i$ とする。近傍点までの平均二乗距離を

$$d_i^2 = \frac{1}{|\mathcal{K}_i|} \sum_{k \in \mathcal{K}_i} \|\mathbf{p}_i^{\text{SfM}} - \mathbf{p}_k^{\text{SfM}}\|_2^2 \quad (101)$$

と定義する。

Gaussian i の初期スケールを

$$\mathbf{s}_i = \sqrt{\max(d_i^2, \varepsilon_s)} \mathbf{1}, \quad \varepsilon_s = 10^{-7} \quad (102)$$

とする。したがって、学習対象となる raw スケールは

$$\tilde{\mathbf{s}}_i = \log \mathbf{s}_i = \frac{1}{2} \log \{\max(d_i^2, \varepsilon_s)\} \mathbf{1} \quad (103)$$

と初期化する。ここで、対数関数はベクトルの各成分に適用する。

回転の初期化

Gaussian の初期回転を単位回転とするため、raw クォータニオンを

$$\tilde{\mathbf{q}}_i = [1 \ 0 \ 0 \ 0]^\top \quad (104)$$

と初期化する。クォータニオンの成分順序は $[q_w, q_x, q_y, q_z]^\top$ とする。

不透明度の初期化

Gaussian の初期不透明度を

$$\alpha_i = \alpha_{\text{init}}, \quad \alpha_{\text{init}} = 0.1 \quad (105)$$

とする。したがって、raw 不透明度は

$$\tilde{\alpha}_i = \text{logit}(\alpha_{\text{init}}) = \log \left(\frac{\alpha_{\text{init}}}{1 - \alpha_{\text{init}}} \right) \quad (106)$$

と初期化する。 $\alpha_{\text{init}} = 0.1$ の場合、 $\tilde{\alpha}_i \simeq -2.1972$ である。

球面調和係数の初期化

0 次の実球面調和基底関数を

$$Y_0^0 = \frac{1}{2\sqrt{\pi}} \simeq 0.282095 \quad (107)$$

とする。式 (10) において、初期状態の Gaussian の色が $\mathbf{c}_i^{\text{SfM}}$ と一致するように、0 次の球面調和係数を

$$\mathbf{h}_{i,00} = \frac{\mathbf{c}_i^{\text{SfM}} - \frac{1}{2}\mathbf{1}}{Y_0^0} \quad (108)$$

と初期化する。それより高次の球面調和係数は

$$\mathbf{h}_{i,lm} = \mathbf{0}, \quad l \geq 1 \quad (109)$$

とする。

球面調和係数は、次数 l の昇順かつ各次数について $m = -l, \dots, l$ の昇順に並べる。 (l, m) に対応する係数のインデックスを

$$b(l, m) = l^2 + l + m \quad (110)$$

とする。このとき、 $l_{\text{max}} = 3$ の場合、Gaussian 一つ当たりの球面調和係数の個数は

$$N_{\text{SH}} = (l_{\text{max}} + 1)^2 = 16 \quad (111)$$

である。

パラメータの保持形式

以上の初期値を用いて、学習パラメータを次の形状の `torch.nn.Parameter` として保持する。

$$\begin{aligned} \mathbf{P}_w &\in \mathbb{R}^{N \times 3}, & \tilde{\mathbf{Q}} &\in \mathbb{R}^{N \times 4}, & \tilde{\mathbf{S}} &\in \mathbb{R}^{N \times 3}, \\ \tilde{\mathbf{A}} &\in \mathbb{R}^{N \times 1}, & \mathbf{H}_{\text{DC}} &\in \mathbb{R}^{N \times 1 \times 3}, & \mathbf{H}_{\text{rest}} &\in \mathbb{R}^{N \times 15 \times 3} \end{aligned}$$

ここで、各テンソルの第 1 次元は Gaussian のインデックスに対応する。 $\mathbf{H}_{\text{DC}}$ には 0 次の球面調和係数を格納し、 $\mathbf{H}_{\text{rest}}$ には 1 次から 3 次までの球面調和係数を格納する。各球面調和係数の第 3 次元は RGB 各成分に対応する。

B.4 参照実装における Gaussian の描画範囲と深度ソート

2D Gaussian は理論上、画像平面全体に非零の値を持つ。しかし、すべての Gaussian について画像全体の画素値を計算すると、計算量が大きくなる。そこで、参照実装では、各 Gaussian の描画範囲を有限の領域に制限する。

描画対象となる Gaussian

Gaussian i のカメラ座標を

$$\mathbf{p}_{c,i} = [x_i \ y_i \ z_i]^\top \quad (112)$$

とする。カメラの後方に存在する Gaussian は画像平面上に投影できないため、

$$z_i > 0 \quad (113)$$

を満たす Gaussian のみを描画対象とする。

また、Gaussian の投影中心 $\mathbf{u}_i$ および描画範囲が画像領域と交差しない場合、その Gaussian を描画対象から除外する。

画像平面上的描画半径

数値安定化を適用した Gaussian i の描画用 2D 共分散行列を

$$\Sigma'_{i,\text{render}} = \begin{bmatrix} a_i & b_i \\ b_i & c_i \end{bmatrix} \quad (114)$$

とする。 $\Sigma'_{i,\text{render}}$ の計算方法は、小節 B.5 に示す。

$\Sigma'_{i,\text{render}}$ の最大固有値は

$$\lambda_{i,\max} = \frac{a_i + c_i + \sqrt{(a_i - c_i)^2 + 4b_i^2}}{2} \quad (115)$$

である。参照実装では、Gaussian の中心から標準偏差の 3 倍までを描画候補とし、画像平面上的描画半径を

$$r_i = \left\lceil 3\sqrt{\max(\lambda_{i,\max}, 0)} \right\rceil \quad (116)$$

とする。

描画矩形

Gaussian i の投影中心を

$$\mathbf{u}_i = \begin{bmatrix} u_{i,x} & u_{i,y} \end{bmatrix}^\top \quad (117)$$

とする。画像の幅を W 、高さを H とすると、Gaussian i の描画矩形を

$$\begin{aligned} x_{i,\min} &= \max(0, \lfloor u_{i,x} - r_i \rfloor), \\ x_{i,\max} &= \min(W - 1, \lceil u_{i,x} + r_i \rceil), \\ y_{i,\min} &= \max(0, \lfloor u_{i,y} - r_i \rfloor), \\ y_{i,\max} &= \min(H - 1, \lceil u_{i,y} + r_i \rceil) \end{aligned} \quad (118)$$

によって定義する。

$x_{i,\min} > x_{i,\max}$ または $y_{i,\min} > y_{i,\max}$ となる場合、Gaussian i の描画矩形は画像領域と交差しないため、その Gaussian を描画対象から除外する。

画像の左上の画素を $(0, 0)$ とし、 x 方向を右向き、 y 方向を下向きとする。画素 j の座標を

$$\begin{aligned} \mathbf{x}_j &= \begin{bmatrix} x_j & y_j \end{bmatrix}^\top, \\ x_j &\in \{0, \dots, W - 1\}, \quad y_j \in \{0, \dots, H - 1\} \end{aligned} \quad (119)$$

とする。

画素 j が Gaussian i の描画矩形に含まれる条件は

$$x_{i,\min} \leq x_j \leq x_{i,\max}, \quad y_{i,\min} \leq y_j \leq y_{i,\max} \quad (120)$$

である。この条件を満たす Gaussian の集合を、画素 j の描画候補集合 $\mathcal{N}_j$ とする。

深度ソート

画素色を前方から後方へ正しく合成するため、描画対象となる Gaussian をカメラ座標における深度 z_i の昇順に並べる。Gaussian k が Gaussian i よりもカメラに近いことを

$$k \prec i \iff z_k < z_i \quad (121)$$

と定義する。

二つの Gaussian の深度が等しい場合は、Gaussian のインデックスが小さいものを手前として扱う。これにより、同一の入力に対して描画順序が一意に定まる。

参照実装では、描画対象となる Gaussian 全体を深度順に並べ、その順序に従って各 Gaussian の描画矩形内の画素を更新する。この処理により、各画素 j に対して $\mathcal{N}_j$ を個別にソートすることなく、同一の深度順序で画素色を合成できる。

画素色の逐次合成

描画対象となる Gaussian を式 (121) の深度順に処理し、各 Gaussian の描画矩形に含まれる画素について、累積色および累積透過率を更新する。不透明度の上限処理、省略判定、累積透過率による処理の打ち切りおよび画素色の逐次合成については、小節 B.6 に示す。

参照実装では、タイル単位のカリングおよびソートを行わず、Gaussian ごとに描画矩形内の画素を処理する。また、深度ソートによって得られる描画順序は離散的な値であるため、逆伝播では描画順序を固定し、ソート処理そのものに関する勾配は計算しない。

B.5 2D 共分散の数値安定化

式 (7) によって得られる 2D 共分散行列は、浮動小数点演算による誤差や、投影後の Gaussian が極端に小さくなることにより、数値的に不安定になる場合がある。そこで、実装では、まず 2D 共分散行列を

$$\Sigma'_{i,\text{sym}} = \frac{1}{2} (\Sigma'_i + \Sigma'^\top_i) \quad (122)$$

と対称化する。その後、対角成分に定数 ε_{cov} を加え、レンダリングに使用する 2D 共分散行列を

$$\Sigma'_{i,\text{render}} = \Sigma'_{i,\text{sym}} + \varepsilon_{\text{cov}} I_2 \quad (123)$$

とする。本実装では、

$$\varepsilon_{\text{cov}} = 0.3 \quad (124)$$

とする。ここで、 ε_{cov} の単位は画素座標系における pixel^2 である。

式 (123) を

$$\Sigma'_{i,\text{render}} = \begin{bmatrix} a_i & b_i \\ b_i & c_i \end{bmatrix} \quad (125)$$

と表し、その行列式を

$$\Delta_i = a_i c_i - b_i^2 \quad (126)$$

とする。逆行列の計算では、丸め誤差によって Δ_i が過度に小さくなることを防ぐため、

$$\Delta_{i,\text{safe}} = \max(\Delta_i, \varepsilon_{\text{det}}), \quad \varepsilon_{\text{det}} = 10^{-8} \quad (127)$$

とする。レンダリングに使用する逆共分散行列は、

$$\Sigma'^{-1}_{i,\text{render}} = \frac{1}{\Delta_{i,\text{safe}}} \begin{bmatrix} c_i & -b_i \\ -b_i & a_i \end{bmatrix} \quad (128)$$

によって計算する。

Gaussian の描画範囲および画素上の不透明度を計算する際は、理論上の Σ'_i ではなく、安定化後の $\Sigma'_{i,\text{render}}$ および $\Sigma'^{-1}_{i,\text{render}}$ を使用する。

参照実装では、式 (122) から式 (128) までの処理は PyTorch のテンソル演算として順伝播に含める。したがって、これらの処理を含む勾配は PyTorch の自動微分によって計算する。

B.6 不透明度合成の数値安定化

Gaussian i の画素 j における不透明度は、安定化後の逆共分散行列を用いて

$$\alpha'_{ij} = \alpha_i \exp \left(-\frac{1}{2} (\mathbf{x}_j - \mathbf{u}_i)^\top \Sigma'^{-1}_{i,\text{render}} (\mathbf{x}_j - \mathbf{u}_i) \right) \quad (129)$$

と計算する。

不透明度が 1 に近い場合、透過率が急激に 0 へ近づき、後続する Gaussian への勾配が消失する可能性がある。そこで、実際に合成に使用する不透明度を

$$\bar{\alpha}'_{ij} = \min(\alpha'_{ij}, \alpha_{\text{max}}), \quad \alpha_{\text{max}} = 0.99 \quad (130)$$

とする。

また、画素色への寄与が十分に小さい Gaussian を省略するため、

$$\hat{\alpha}'_{ij} = \begin{cases} \bar{\alpha}'_{ij}, & \bar{\alpha}'_{ij} \geq \alpha_{\min}, \\ 0, & \bar{\alpha}'_{ij} < \alpha_{\min}, \end{cases} \quad \alpha_{\min} = \frac{1}{255} \quad (131)$$

とする。

画素 j に重なる Gaussian をカメラから近い順に $1, \dots, M_j$ とする。画素色の累積値を $C_{0,j} = \mathbf{0}$ 、透過率を $T_{0,j} = 1$ とする。

i 番目の Gaussian を合成する前に、合成後の候補透過率を

$$T_{i,j}^{\text{test}} = T_{i-1,j} (1 - \hat{\alpha}'_{ij}) \quad (132)$$

と計算する。ここで、

$$T_{i,j}^{\text{test}} < T_{\min}, \quad T_{\min} = 10^{-4} \quad (133)$$

となった場合は、 i 番目の Gaussian を画素色へ合成せず、 i 番目以降の Gaussian の処理を終了する。

一方、 $T_{i,j}^{\text{test}} \geq T_{\min}$ である場合は、累積色および累積透過率を

$$\mathbf{C}_{i,j} = \mathbf{C}_{i-1,j} + T_{i-1,j} \hat{\alpha}'_{ij} \mathbf{c}_i, \quad (134)$$

$$T_{i,j} = T_{i,j}^{\text{test}} \quad (135)$$

と更新し、次の Gaussian の処理へ進む。

実際に画素色へ合成された最後の Gaussian の添字を K_j とする。すべての Gaussian が合成された場合は $K_j = M_j$ とし、一つも合成されなかった場合は $K_j = 0$ とする。

背景色を $\mathbf{c}_{\text{bg}}$ とすると、最終的な画素色は

$$\boldsymbol{\gamma}_j = \mathbf{C}_{K_j,j} + T_{K_j,j} \mathbf{c}_{\text{bg}} \quad (136)$$

である。背景色 $\mathbf{c}_{\text{bg}}$ はデータセットに応じて設定する。原論文の再現実験では、実世界シーンでは黒背景を使用し、Synthetic NeRF では白背景を使用する。

不透明度の下限判定および透過率による打ち切り判定は離散的な処理であるため、これらの判定条件自体に関する勾配は計算しない。参照実装では、判定によって合成対象となった Gaussian について、式 (130) から累積色および累積透過率の更新までのテンソル演算を PyTorch の自動微分の対象とする。

B.7 球面調和関数の次数および学習解像度の段階的増加

球面調和係数の最適化は、学習画像に含まれる視線方向の情報が十分でない場合に不安定になる可能性がある。そこで、本実装では、学習開始時には0次の球面調和関数のみを使用し、学習の進行に伴って使用する最大次数を段階的に増加させる。

また、学習初期の最適化を安定させるため、学習画像の解像度も段階的に増加させる。¹⁷⁾

B.8 ハイパーパラメータ

中心位置の学習率スケジュール

中心位置 $p_{w,i}$ の学習率は、学習の進行に伴って初期学習率 $\eta_{p,\text{init}}$ から最終学習率 $\eta_{p,\text{final}}$ まで指数的に減衰させる。総反復回数を T 、現在の反復回数を t として、基準となる中心位置の学習率を

$$\bar{\eta}_p(t) = \exp \left[\left(1 - \frac{t}{T} \right) \log \eta_{p,\text{init}} + \frac{t}{T} \log \eta_{p,\text{final}} \right] \quad (137)$$

とする。

実際に使用する中心位置の学習率は、scene extent を R_{scene} として

$$\eta_p(t) = R_{\text{scene}} \bar{\eta}_p(t) \quad (138)$$

とする。¹⁸⁾ 総反復回数 T は、対象とするデータおよび実験条件に応じて設定する。¹⁹⁾ 中心位置以外の学習パラメータについては、表2に示す一定の学習率を使用する。

本実装で使用するハイパーパラメータを表2に示す。

B.9 実装の検証

実装の妥当性を確認するため、以下の項目を検証する。

1. 単一の Gaussian を光軸上に配置したとき、レンダリング画像の中心に楕円状の分布が描画されること。
2. 等方的なスケールを持つ Gaussian について、クォータニオンを変化させてもレンダリング結果が変化しないこと。

17) 原論文の再現実験では、球面調和関数の最大次数を1000反復ごとに1ずつ増加させ、最大3次まで使用する。また、250反復目までは元画像の幅および高さをそれぞれ1/4に縮小した画像を使用し、251反復目から500反復目までは幅および高さをそれぞれ1/2に縮小した画像を使用する。それ以降は元の解像度を使用する。

18) ここで、 $\eta_{p,\text{init}}$ は $t=0$ に対応する基準値であり、実際の学習では反復番号を1始まりとするため、最初に使用する学習率は $\eta_p(1)$ である。

19) 原論文の再現実験では、 $\eta_{p,\text{init}} = 1.6 \times 10^{-4}$ 、 $\eta_{p,\text{final}} = 1.6 \times 10^{-6}$ 、 $T = 30000$ とする。

3. 3D 共分散行列および 2D 共分散行列が対称正定値行列となること。
4. 参照実装において、各学習パラメータについて、PyTorch の自動微分によって得られた勾配と数値微分によって得られた勾配が一致すること。
5. 固定した単一学習視点を1,000反復学習し、最初の50反復における平均損失に対して、最後の50反復における平均損失が50%未満となることを確認する。
6. 順伝播および逆伝播の計算結果に NaN または Inf が含まれないこと。

C Gaussian の適応的密度制御

3D Gaussian Splatting では、学習中に Gaussian のパラメータを更新するだけでなく、Gaussian の clone、split および prune を行う。これにより、初期の SfM 点群では十分に表現できない領域に Gaussian を追加するとともに、レンダリングへの寄与が小さい Gaussian や過度に大きい Gaussian を除去する。この処理を適応的密度制御と呼ぶ。

Gaussian i について、学習中に得られた画像平面上の位置勾配の平均を

$$g_i = \frac{1}{|\mathcal{O}_i|} \sum_{t \in \mathcal{O}_i} \left\| \frac{\partial \mathcal{L}^{(t)}}{\partial \mathbf{u}_i^{(t)}} \right\|_2 \quad (139)$$

とする。ここで、 $\mathcal{O}_i$ は現在の統計量の蓄積期間において Gaussian i が観測された学習反復の集合、 $\mathcal{L}^{(t)}$ は第 t 反復において計算された損失、 $\mathbf{u}_i^{(t)}$ は第 t 反復における Gaussian i の画像平面上の投影座標である。

g_i が所定の閾値以上である Gaussian は densification の対象となる。ただし、Gaussian の大きさに応じて clone または split を使い分ける。

C.1 Gaussian の clone

位置勾配が大きく、かつ三次元空間上で小さい Gaussian は clone の対象とする。Gaussian i の最大スケールを

$$s_{i,\text{max}} = \max(s_{i,x}, s_{i,y}, s_{i,z}) \quad (140)$$

とする。

位置勾配およびスケールが

$$g_i \geq \tau_g, \quad s_{i,\text{max}} \leq \tau_s \quad (141)$$

を満たす場合、Gaussian i を clone する。

clone によって生成される Gaussian は、元の Gaussian と同一の中心位置、スケール、不透明度、クォータニオンおよび球面調和係数を用いて初期化する。その後の勾配更新によって、元の Gaussian と clone された Gaussian が独立に最適化される。

C.2 Gaussian の split

位置勾配が大きく、かつ三次元空間上で大きい Gaussian は split の対象とする。すなわち、

$$g_i \geq \tau_g, \quad s_{i,\max} > \tau_s \quad (142)$$

を満たす Gaussian i を split する。

split では、元の Gaussian から 2 個の子 Gaussian を生成する。各子 Gaussian の中心位置は、元の Gaussian のスケールおよび回転によって定まる三次元分布からサンプリングした変位を元の中心位置に加えることで決定する。

また、子 Gaussian のスケールは、元の Gaussian のスケールを

$$s_{i,\text{child}} = \frac{s_i}{1.6} \quad (143)$$

として縮小する。不透明度、クォータニオンおよび球面調和係数は元の Gaussian から継承する。子 Gaussian を生成した後、元の Gaussian は除去する。

C.3 Gaussian の prune

レンダリングへの寄与が小さい Gaussian や、過度に大きい Gaussian は prune によって除去する。

Gaussian i の不透明度が

$$\alpha_i < \tau_\alpha \quad (144)$$

を満たす場合、その Gaussian を除去する。

また、Gaussian の画像平面上における最大投影半径を $r_{i,\max}$ とすると、

$$r_{i,\max} > \tau_r \quad (145)$$

を満たす Gaussian も除去対象とする。

さらに、三次元空間上の最大スケール $s_{i,\max}$ が所定の閾値を超える Gaussian も除去する。

C.4 適応的密度制御の実行

本実装では、各 Gaussian について画像平面上の位置勾配の累積値、位置勾配が観測された回数、および画像平面上の最大投影半径を保持する。位置勾配の平均 g_i は、累積した位置勾配を観測回数で除することで計算する。

原論文の再現実験では、適応的密度制御に用いる位置勾配の閾値を

$$\tau_g = 2.0 \times 10^{-4} \quad (146)$$

とする。

また、Gaussian を clone するか split するかを判定する三次元スケールの閾値は、シーンの空間的な広がりを表す scene extent を R_{scene} として

$$\tau_s = 0.01 R_{\text{scene}} \quad (147)$$

とする。

適応的密度制御では、学習中に各 Gaussian について統計量を蓄積し、あらかじめ設定した実行期間中に、一定の反復間隔で clone、split および prune を行う。各適応的密度制御の実行後には、位置勾配の累積値、観測回数および最大投影半径をリセットし、次の適応的密度制御に向けて再び統計量を蓄積する。²⁰⁾

C.5 不透明度のリセット

学習中に Gaussian の不透明度が過度に大きくなることを防ぐため、所定の反復において不透明度をリセットする。本実装では、各 Gaussian の不透明度を

$$\alpha_i \leftarrow \min(\alpha_i, \alpha_{\text{reset}}) \quad (148)$$

によって制限する。²¹⁾

D 4D Gaussian Splatting への拡張

本実装では、3D Gaussian Splatting を動的シーンへ拡張するため、時系列画像の各フレームに対して 3D Gaussian を逐次的に最適化する。このとき、各フレームを独立に初期化するのではなく、一つ前のフレームで最適化した Gaussian を次のフレームの初期値として用いる。

フレーム番号を $f = 1, \dots, F$ とし、フレーム f における Gaussian i のパラメータを

$$\theta_i^{(f)} = \left[p_{w,i}^{(f)\top} \quad q_i^{(f)\top} \quad s_i^{(f)\top} \quad \alpha_i^{(f)} \quad h_i^{(f)\top} \right]^\top \quad (149)$$

とする。

最初のフレーム $f = 1$ では、3D Gaussian Splatting と同様に SfM 点群から Gaussian を初期化し、対象フレームの画像を用いて各 Gaussian のパラメータを最適化する。

20) 原論文の再現実験では、500 反復目より後から 15000 反復目より前まで、100 反復ごとに適応的密度制御を実行する。

21) 原論文の再現実験では、 $\alpha_{\text{reset}} = 0.01$ とし、3000 反復ごとに不透明度をリセットする。

一方、 $f \geq 2$ については、一つ前のフレーム $f-1$ において最適化された Gaussian のパラメータを、フレーム f の学習開始時の初期値として

$$\theta_i^{(f,0)} = \theta_i^{(f-1,*)} \quad (150)$$

と設定する。ここで、 $\theta_i^{(f,0)}$ はフレーム f における最適化前のパラメータ、 $\theta_i^{(f-1,*)}$ はフレーム $f-1$ における最適化後のパラメータである。²²⁾

したがって、各パラメータについて

$$\begin{aligned} \mathbf{p}_{w,i}^{(f,0)} &= \mathbf{p}_{w,i}^{(f-1,*)}, \\ \mathbf{q}_i^{(f,0)} &= \mathbf{q}_i^{(f-1,*)}, \\ \mathbf{s}_i^{(f,0)} &= \mathbf{s}_i^{(f-1,*)}, \\ \alpha_i^{(f,0)} &= \alpha_i^{(f-1,*)}, \\ \mathbf{h}_i^{(f,0)} &= \mathbf{h}_i^{(f-1,*)} \end{aligned} \quad (151)$$

とする。

フレーム間でシーンの変化が十分に小さい場合、隣接するフレームにおける Gaussian の位置、形状、色および不透明度も大きく変化しないと考えられる。そのため、一つ前のフレームの学習結果を初期値として利用することで、各フレームを SfM 点群から独立に初期化する場合と比較して、次フレームにおける最適化を効率化できると考えられる。

適応的密度制御によって Gaussian の総数が変化した場合には、フレーム $f-1$ の学習終了時に存在する Gaussian 集合全体をフレーム f の初期 Gaussian 集合として引き継ぐ。したがって、フレーム間で Gaussian の総数も継承される。

本稿では、まず以上の方法によって時間方向へ Gaussian を引き継ぐことで、3D Gaussian Splatting を動的シーンへ拡張する。²³⁾

22) 一方、Adam が保持する一次モーメントおよび二次モーメントはフレームごとにリセットする。また、適応的密度制御に用いる位置勾配の累積値、観測回数および最大投影半径についても、次フレームの学習開始時にリセットする。これにより、前フレームで得られた Gaussian を初期値として利用しつつ、各フレームの損失関数に対する最適化を独立に行う。

23) Gaussian の時間変化を MLP などによってモデル化する方法については、今後の拡張として検討する。

表 2: 本実装および原論文再現実験で使用する主要なハイパーパラメータ

分類	パラメータ	設定値	説明
基本設定			
基本設定	数値精度	float32	Gaussian のパラメータおよび画像を 32 ビット浮動小数点数で保持する。
基本設定	計算デバイス	cuda	原論文の再現実験では GPU を用いて学習およびレンダリングを行う。
基本設定	乱数シード	0	初期化および学習視点の選択に用いる乱数を固定する。
基本設定	初期 Gaussian 数	$N = N_{\text{SfM}}$	実世界データでは、SfM 点 1 点につき一つの Gaussian を生成する。Synthetic NeRF を用いた再現実験では 100000 個の Gaussian から学習を開始する。
基本設定	学習中の Gaussian 数	動的	適応的密度制御による clone、split および prune によって学習中に変化する。
基本設定	背景色	データ依存	原論文の再現実験では、実世界シーンに黒背景、Synthetic NeRF に白背景を使用する。
基本設定	球面調和関数の最大次数	$l_{\max} = 3$	学習開始時には 0 次のみを使用し、学習中に最大次数を段階的に増加させる。
基本設定	適応的密度制御	有効	学習中に Gaussian の clone、split および prune を行う。
基本設定	不透明度のリセット	有効	学習中に一定間隔で Gaussian の不透明度を小さい値に制限する。
基本設定	SH 次数の段階的増加	有効	学習開始時には 0 次のみを使用し、最大 3 次まで段階的に増加させる。
基本設定	学習解像度の段階的増加	有効	学習初期には低解像度画像を使用し、その後フル解像度へ段階的に増加させる。
Gaussian の初期化			
初期化	スケール計算の近傍点数	3	各初期点に最も近い 3 点までの平均二乗距離を使用する。
初期化	平均二乗距離の下限	$\varepsilon_s = 10^{-7}$	初期スケールが 0 となることを防ぐ。
初期化	初期不透明度	$\alpha_{\text{init}} = 0.1$	raw 不透明度は $\tilde{\alpha}_i = \logit(0.1)$ と初期化する。
初期化	初期クォータニオン	$[1, 0, 0, 0]^\top$	Gaussian の初期回転を単位回転とする。
初期化	高次 SH 係数の初期値	0	$l \geq 1$ の球面調和係数を 0 とする。
初期化	クォータニオン正規化の下限	$\varepsilon_q = 10^{-8}$	除算時には $\max(\ \tilde{\mathbf{q}}_i\ _2, \varepsilon_q)$ を分母として使用する。
レンダリングおよび数値安定化			
レンダリング	レンダリング実装	参照 / CUDA	数式および勾配の検証には PyTorch による参照実装を使用し、原論文の再現実験には公式 CUDA rasterizer を使用する。
レンダリング	Gaussian の描画範囲	3σ	参照実装では、2D 共分散行列の最大固有値に基づく半径 $r_i = \lceil 3\sqrt{\lambda_{i,\max}} \rceil$ の範囲を描画候補とする。
レンダリング	深度条件	$z_i > 0$	カメラの後方にある Gaussian を描画対象から除外する。
レンダリング	深度ソート	手前から奥	Gaussian をカメラから近い順に処理して前方から後方へ合成する。
レンダリング	タイル処理	実装依存	参照実装ではタイル処理を行わない。CUDA 実装では公式実装の tile-based rasterizer を使用する。
数値安定化	2D 共分散の低域通過項	$\varepsilon_{\text{cov}} = 0.3$	2D 共分散行列の対角成分に 0.3 を加える。
数値安定化	行列式の下限	$\varepsilon_{\text{det}} = 10^{-8}$	参照実装で 2D 共分散行列の逆行列を計算する際の数値的不安定性を防ぐ。
数値安定化	画素不透明度の上限	$\alpha_{\max} = 0.99$	画素ごとの不透明度を $\min(0.99, \alpha'_{ij})$ に制限する。
数値安定化	不透明度の省略閾値	$\alpha_{\min} = 1/255$	$\alpha'_{ij} < 1/255$ となる Gaussian の寄与を無視する。
数値安定化	累積透過率の終了閾値	$T_{\min} = 10^{-4}$	累積透過率が閾値未満となった時点で、それより奥の合成を終了する。
適応的密度制御			
密度制御	位置勾配の閾値	$\tau_g = 2.0 \times 10^{-4}$	clone または split を行う Gaussian を位置勾配に基づいて判定する。

次ページに続く

表 2 の続き

分類	パラメータ	設定値	説明
密度制御	densification のスケール閾値	$0.01R_{\text{scene}}$	最大スケールが閾値以下の場合は clone、閾値より大きい場合は split の対象とする。
密度制御	prune の不透明度閾値	0.005	不透明度が 0.005 未満の Gaussian を除去する。
密度制御	画像平面上の半径閾値	20	画像平面上で過度に大きい Gaussian を prune する際の閾値として使用する。
密度制御	三次元スケールの prune 閾値	$0.1R_{\text{scene}}$	三次元空間上で過度に大きい Gaussian を除去する際に使用する。
密度制御	densification 開始反復	500	原論文の再現実験では、500 反復目より後から密度制御イベントを実行する。
密度制御	densification 終了反復	15000	原論文の再現実験では、15000 反復目より前まで密度制御イベントを実行する。
密度制御	densification 間隔	100	原論文の再現実験では 100 反復ごとに clone、split および prune を行う。
密度制御	不透明度リセット間隔	3000	学習中に 3000 反復ごとに不透明度をリセットする。
密度制御	リセット後の最大不透明度	0.01	不透明度をリセットする際、各 Gaussian の不透明度を最大 0.01 に制限する。
学習スケジュール			
学習スケジュール	SH 次数の増加間隔	1000	1000 反復ごとに使用する球面調和関数の最大次数を 1 ずつ増加させ、最大 3 次まで使用する。
学習スケジュール	初期解像度	1/4	250 反復目までは元画像の 1/4 の解像度を使用する。
学習スケジュール	中間解像度	1/2	251 反復目から 500 反復目までは元画像の 1/2 の解像度を使用する。
学習スケジュール	最終解像度	1	500 反復目より後は元画像の解像度を使用する。
損失関数			
損失関数	D-SSIM 損失の重み	$\lambda = 0.2$	全体の損失を $\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}}$ とする。
SSIM	局所窓の大きさ	11×11	各画素を中心とする局所領域として使用する。
SSIM	ガウス窓の標準偏差	1.5	SSIM の局所統計量を計算するガウス窓に使用する。
SSIM	安定化定数	$K_1 = 0.01$	SSIM の輝度項に使用する。
SSIM	安定化定数	$K_2 = 0.03$	SSIM のコントラストおよび構造項に使用する。
最適化			
最適化	最適化手法	Adam	すべての学習パラメータを Adam によって更新する。
最適化	総反復回数	実験依存	対象とするデータおよび実験条件に応じて設定する。原論文の再現実験では 30000 反復とする。
最適化	1 反復当たりの学習視点数	1	各反復で一つの学習視点を選択する。
最適化	学習視点の選択方法	ランダム	全学習視点をランダムな順序で重複なく選択し、すべて選択した後に再度順序をランダム化する。
Adam	一次モーメント係数	$\beta_1 = 0.9$	Adam の一次モーメント推定に使用する。
Adam	二次モーメント係数	$\beta_2 = 0.999$	Adam の二次モーメント推定に使用する。
Adam	安定化定数	$\epsilon_{\text{Adam}} = 10^{-15}$	Adam の更新式におけるゼロ除算を防ぐ。
Adam	重み減衰	0	本実装では重み減衰を使用しない。
学習率	中心位置の初期学習率	1.6×10^{-4}	中心位置の学習率スケジュールに使用する基準値である。実際の学習率には scene extent を乗じる。
学習率	中心位置の最終学習率	1.6×10^{-6}	中心位置の学習率スケジュールに使用する基準値である。実際の学習率には scene extent を乗じる。
学習率	0 次 SH 係数	2.5×10^{-3}	球面調和関数の 0 次係数に使用する。
学習率	高次 SH 係数	1.25×10^{-4}	0 次係数の学習率の 1/20 とする。
学習率	raw 不透明度	5.0×10^{-2}	raw 不透明度 $\tilde{\alpha}_i$ に使用する。
学習率	raw スケール	5.0×10^{-3}	raw スケール $\tilde{s}_i$ に使用する。
学習率	raw クォータニオン	1.0×10^{-3}	raw クォータニオン $\tilde{q}_i$ に使用する。
評価			

次ページに続く

表 2 の続き

分類	パラメータ	設定値	説明
評価	画素値域	$[0, 1]$	評価時にはレンダリング画像の RGB 値を $[0, 1]$ へクリップする。
評価	PSNR の最大画素値	$I_{\max} = 1$	各テスト画像について PSNR を計算し、その算術平均を報告する。
評価	評価指標	PSNR, SSIM, LPIPS	原論文との比較では PSNR、SSIM および LPIPS を用いる。